

# MANE - 4500 Modeling and Control of Dynamical Systems

## The Nacho Supreme - Part #2

Quinten Pienaar 662041924

### I. Overview

This paper is a proposed control system for the Nacho Supreme self balancing toy. The toy is a right isosceles triangular prism. The dimensions of the triangle are 3x3x4.3 inches, and it is 1.5 inches thick. A Nidec smart 20N brushless dc motor powered by a 1s 4000mAh LiPo battery powers a wheel spoked reaction wheel with a 1.5 inch outer diameter and a 1 inch inner diameter. The motor is equipped with a current sensor, and the chassis angle off of vertical is measured with an IMU unit. Figure 1 below depicts a Speed, Current, Efficiency and Power curve for the dc motor. Figure 2 shows the control block diagram.

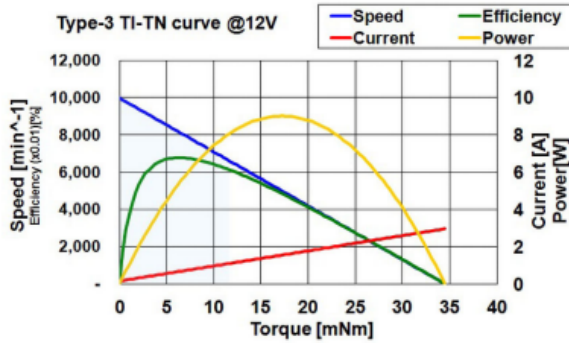


Fig. 1 Speed Current Efficiency Power Curve (adapted from [1])

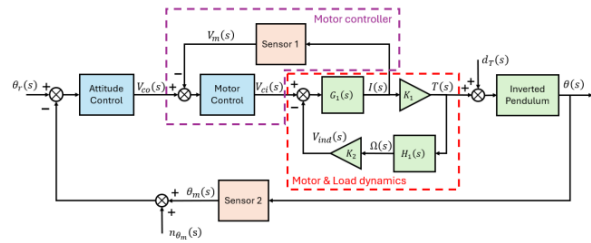


Fig. 2 Control System Block Diagram (adapted from [1])

### II. Analysis

Before a control system could be designed, transfer functions for blocks in green and pink had to be determined. Blocks in green were determined through the physical dynamics of the motor, reaction wheel and overall chassis. Blocks in pink were determined by generating bode diagrams. Table one below outlines relevant variables for the toys dynamics.

| Assembly parameters                        |                                   |                                            |                          |
|--------------------------------------------|-----------------------------------|--------------------------------------------|--------------------------|
| Center of gravity ( $Y_{CG}$ )             | Mass ( $M_{asm}$ )                | Rotational Inertia ( $J_{asm}$ )           |                          |
| 3.81 [cm]                                  | 0.178 [kg]                        | $3.53 \times 10^{-4}$ [kg·m <sup>2</sup> ] |                          |
| DC-motor parameters                        |                                   |                                            |                          |
| Rotational Inertia ( $J_{dcm}$ )           | Viscous Damping ( $D_{dcm}$ )     | Resistance ( $R_{dcm}$ )                   | Inductance ( $L_{dcm}$ ) |
| $3.7 \times 10^{-8}$ [kg·m <sup>2</sup> ]  | $2.0 \times 10^{-4}$ [N·m·s/rad]  | 4 [ $\Omega$ ]                             | 0.04 [mH]                |
| Reaction Wheel parameters                  |                                   |                                            |                          |
| Rotational Inertia ( $J_{rw}$ )            | Viscous damping ( $D_{rw}$ )      |                                            |                          |
| $2.62 \times 10^{-6}$ [kg·m <sup>2</sup> ] | $3.76 \times 10^{-4}$ [N·m·s/rad] |                                            |                          |

Table 1 System Parameters (adapted from [1])

### A. Motor and Load Dynamics

In determining the transfer functions for the motor and load dynamics section it's helpful to outline the input and output for each block. G1 is the electrical dynamics of the motor, its input is volts and its output is current. K1 is the torque constant, its input is current and its output is torque. H1 is the reaction wheel dynamics, its input is torque and its output is rotational speed. K2 is the back emf constant, its input is rotational speed and its output is voltage. G1 is given below.

$$G_1 = \frac{\tau}{V} = \frac{1}{L_{dcm}s + R} = \frac{1}{0.04 * 10^{-3}s + 4} = \frac{25000}{s + 1 * 10^5} \quad (1)$$

In order to determine K1, the stall current of the motor was found from figure 1. The point on the graph where the Speed curve is equal to 0 corresponds to a torque of 34.7 mNm and a current of roughly 3 amps. Therefore:

$$K_1 = \frac{\tau_{stall}}{I_{stall}} = \frac{34.7 * 10^{-3} Nm}{3A} = 0.0116 \frac{Nm}{A} \quad (2)$$

H1 represents the physical dynamics of both the DC motor's fly wheel and the reaction wheel.

$$H_1 = \frac{\omega}{\tau} = \frac{1}{(J_{dcm} + J_{rw})s + (D_{dcm} + D_{rw})} = \frac{1}{J_{eq}s + D_{eq}} = \frac{1}{2.6570 * 10^{-6}s + 5.760 * 10^{-4}} = \frac{3.7636 * 10^5}{s + 216.8} \quad (3)$$

K2, the back emf constant was determined by dividing the voltage output of the motor by the rotational speed of the motor with no load connected to it in radians per second. Our motor is specified to output 12 volts, and from figure 1  $\omega_{noload}$  was determined as 10000 rpm.

$$K_2 = \frac{V}{\omega_{noload}} = \frac{12V}{\frac{2\pi}{60} * 10000 \frac{rad}{s}} = 0.0115 \frac{Vs}{rad} \quad (4)$$

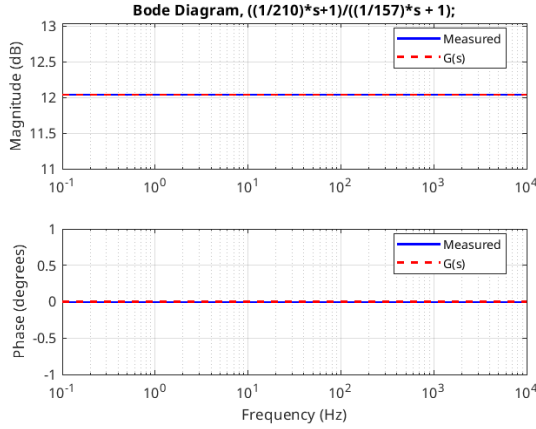
### B. Inverted Pendulum

The inverted Pendulum transfer function takes torque as an input and outputs an angle off of vertical. The rotational inertia is the sum of the rotational inertia of the assembly, dc motor and reaction wheel. The net torque is given by the torque generated from the reaction wheel and any disturbances on the system. The damping is the net damping on the motor and reaction wheel.

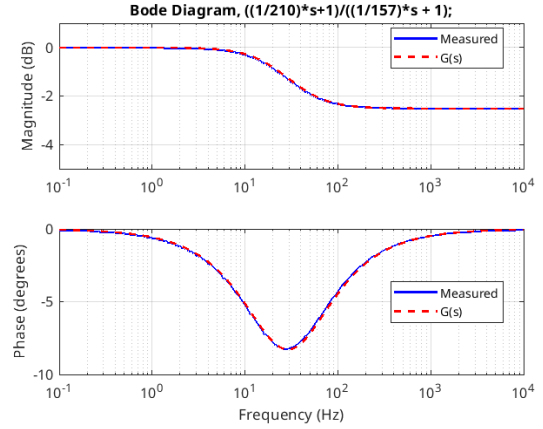
$$G_2 = \frac{\theta}{\tau_{rw} + \tau_{disturbance}} = \frac{1}{(J_{asm} + J_{rw} + J_{dcm})s^2 + (D_{rw} + D_{dcm})} = \frac{1}{J_{sys}s^2 + D_{sys}} = \frac{2832.9}{s^2 - 189.4} \quad (5)$$

## III. Sensor 1 and 2

Sensors 1 and 2 transfer functions were both determined by numerically generating bode diagrams. Sensor 1 is the current sensor and sensor 2 is the IMU. Both sensors were given as protected matlab functions. Their inputs are a time array and a forcing function and they create an output array. By iteratively creating input sinusoidal functions at a variety of frequencies and then saving output data, we could create bode diagrams. The code for these diagrams is given in Appendix A. The bode diagrams are shown in the figures below. The theoretical transfer function is overlaid on each diagram.



**Fig. 3 Sensor 1 Bode Diagram**



**Fig. 4 Sensor 2 Bode Diagram**

The transfer function of sensor 1 was determined by solving the equation  $20\log_{10}(|k|) = 12.04$ . For sensor 2, based on the fact that the magnitude starts at 0 db, it is assumed that  $k = 1$ . The negative slope indicates the presence of a real pole, but the fact that the diagram soon after levels out indicates the presence of a real zero. Therefore, the form  $G = \frac{zero*s+1}{pole*s+1}$  was entered into matlab, and then different numbers were tried until a transfer function could be approximated. The final transfer functions are as follows.

$$G_{sensor1} = 4 \quad (6)$$

$$G_{sensor2} = \frac{0.74822(s + 210.5)}{s + 157.5} \quad (7)$$

#### IV. Control

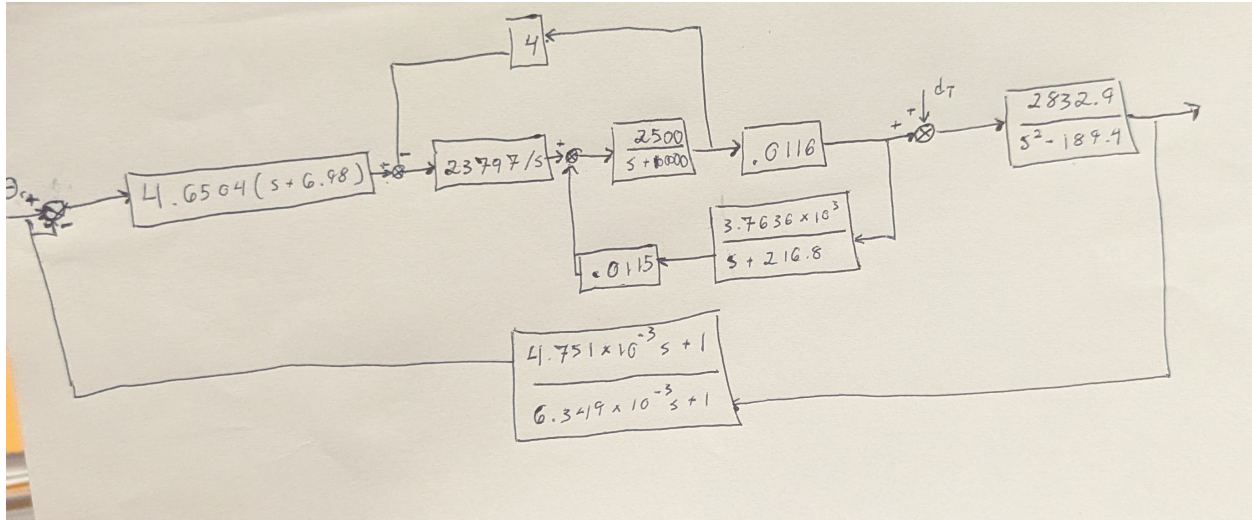
Using the siso tool, the transfer function for the motor controller and attitude controller were designed such that the entire system has a settling time of between 0.5 and 3 seconds, and an overshoot of less than 15%. This was done with success, and the motor controller transfer function was determined to be

$$C_m = 1.5329 \quad (8)$$

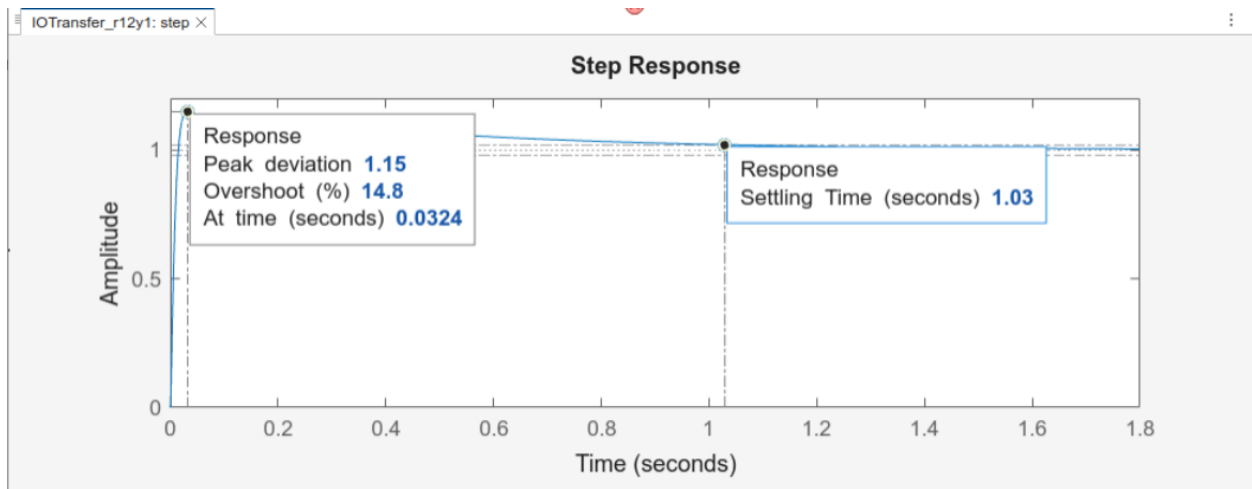
The transfer function for Attitude Control was determined to be

$$C_a = 4.2515(s + 16.11) \quad (9)$$

Using the above controllers, an overshoot of 14.8% and a settling time of 1.03 seconds was achieved. The final block diagram and step response is given in the figure below. Additionally the root locus, and bode diagrams of the entire system are given in Appendix B. The code used to create these graphs are given in Appendix A.



**Fig. 5 Final block diagram**



**Fig. 6 Sisotool step response**

## V. Conclusion

Overall a successful control system was designed. The system had a small settling time, overshoot, and steady state error. However the small settling time raises eye brows. Additionally, there was some discrepancy from the step response outputted by the sisotool and that generated by a matlab script. That being said, both systems are stable with small overshoots, the only real difference is the settling time. Overall the system seems to be successful, and will generally output reasonable actuation signals due to relatively small gain values. The biggest concern is the difference between the sisotool step response metrics and those calculated with the matlab script in Appendix A.

## Appendix

### A. Matlab code

```
1 %intialize
2 fexp = -1:.01:4;
3 f = 10.^fexp;
4 magnitude = zeros(size(f));
5 phase = zeros(size(f));
6 for i = 1:length(f)
7     %collect data
8     T = 1/f(i);
9     t = 0:T/10000:20*T;
10    in = sin(2*pi*f(i)*t);
11    out = sensor1(in, t);
12    %find output peaks
13    [pkso, locso] = findpeaks(out);
14    [pksi, locsi] = findpeaks(in);
15    Mlast_outpeak = pkso(end); Tlast_outpeak = locso(end);
16    Mlast_inpeak = pksi(end); Tlast_inpeak = locsi(end);
17    %calculate magnititude and phase shift
18    magnitude(i) = 20*log10(Mlast_outpeak/Mlast_inpeak);
19    dt = (t(Tlast_inpeak)-t(Tlast_outpeak));
20    phase(i) = dt*f(i)*360;
21 end
22 %%
23 s = tf('s');
24 % k = 1;
25 % G = k * ((1/210.5)*s+1)/((1/157.5)*s + 1);
26 G = tf(4, 1);
27
28 % Evaluate Bode data of G over your frequency range
29 w = 2*pi*f; % convert Hz to rad/s
30 [magG, phaseG] = bode(G, w);
31 magG = squeeze(magG); % magnitude in absolute
32 phaseG = squeeze(phaseG); % phase in degrees
33 magG_dB = 20*log10(magG); % convert to dB
34
35 % Plot both Bode diagrams
36 figure;
37
38 % Magnitude plot
39 subplot(2,1,1);
40 semilogx(f, magnitude, 'b', 'LineWidth', 1.5); hold on;
41 semilogx(f, magG_dB, 'r--', 'LineWidth', 1.5);
42 ylim([-5 1])
43 grid on;
44 ylabel('Magnitude (dB)');
45 title('Bode Diagram, ((1/210)*s+1)/((1/157)*s + 1);');
46 legend('Measured', 'G(s)', 'Location', 'Best');
47
48 % Phase plot
49 subplot(2,1,2);
50 semilogx(f, phase, 'b', 'LineWidth', 1.5); hold on;
51 semilogx(f, phaseG, 'r--', 'LineWidth', 1.5);
52 grid on;
53 ylabel('Phase (degrees)');
54 xlabel('Frequency (Hz)');
55 legend('Measured', 'G(s)', 'Location', 'Best');
```

**Listing 1 MATLAB code to create bode diagrams for sensor 1 and 2**

```

1 %motor & load dynamics
2 s = tf('s');
3 G1 = 1/(.04e-3*s + 4);
4 K1 = 34.7e-3/3;
5 H1 = 1/((3.7e-8+2.62e-6)*s + (2e-4+3.76e-4));
6 K2 = 12/(2*pi*10000/60);
7
8 %general motor dynamics tf
9 Gm = G1*K1/(1+G1*K1*H1*K2);
10 %sensor 1 -> divided by k1 due to moving pickoff point upstream
11 sensor1 = 4/K1;
12 %inverted pendulum
13 Gp = 2833/(s^2-189.4);
14 %sensor 2
15 sensor2 = ((1/210.5)*s+1)/((1/157.5)*s + 1);
16 %Motor controller
17 C2 = 23797/s;
18 %Attitude Control
19 C1 = 4.6504*(s+6.98);
20
21 %inner loop
22 Gloop1 = C2*Gm/(1+C2*Gm*sensor1);
23 %system
24 overall_system = C1*Gloop1*Gp/(1+C1*Gloop1*Gp*sensor2);
25 %plotting
26 step(overall_system)
27 title('Step Response from Script')
28 info = stepinfo(overall_system)
29 %%
30 figure(2)
31 bode(overall_system)
32 title('System Bode Diagram - Quinten Pienaar')
33 grid on
34 %%
35 figure(3)
36 rlocus(overall_system);
37 title('Root Locus Plot');
38 xlabel('Real Axis');
39 ylabel('Imaginary Axis');
40 grid on;

```

**Listing 2 MATLAB code for diagrams relating to stability of overall system**

```

1 >> motor_controller
2
3 info =
4
5 struct with fields:
6
7     RiseTime: 0.0143
8     TransientTime: 0.0218
9     SettlingTime: 0.0218
10    SettlingMin: 1.0232
11    SettlingMax: 1.1478
12    Overshoot: 1.0088

```

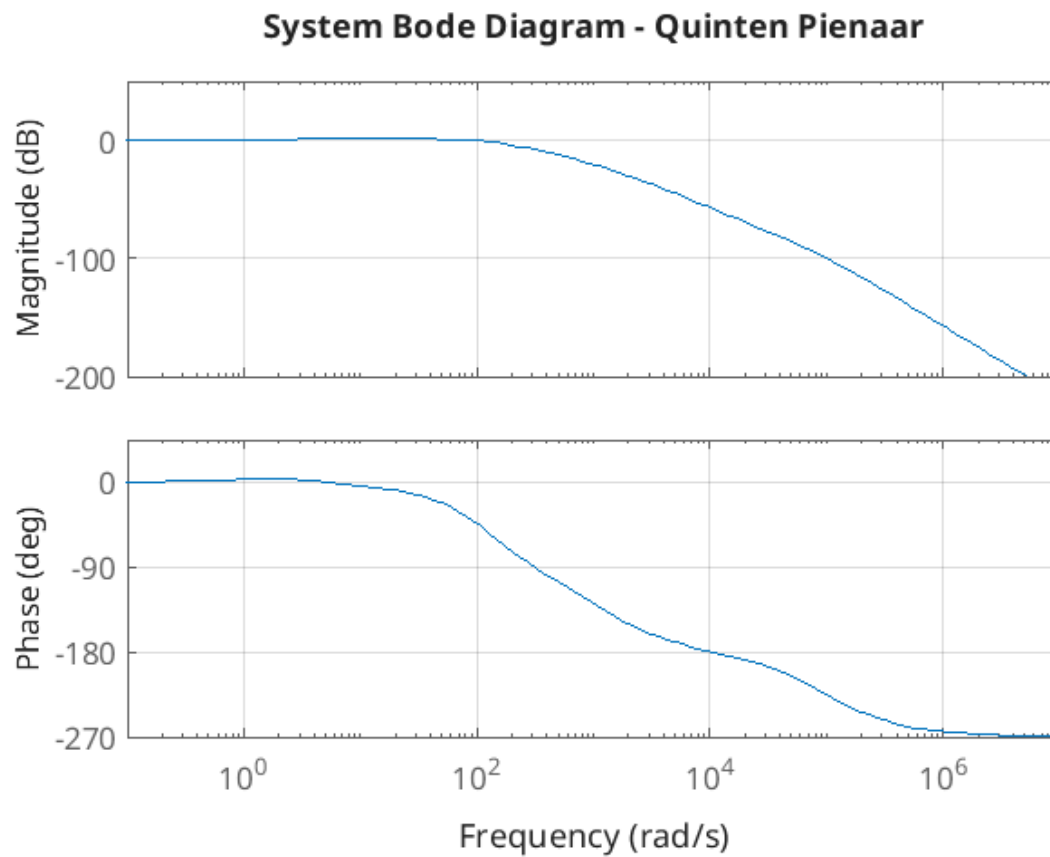
```

13     Undershoot: 0
14     Peak: 1.1478
15     PeakTime: 0.0327

```

**Listing 3** info regarding system

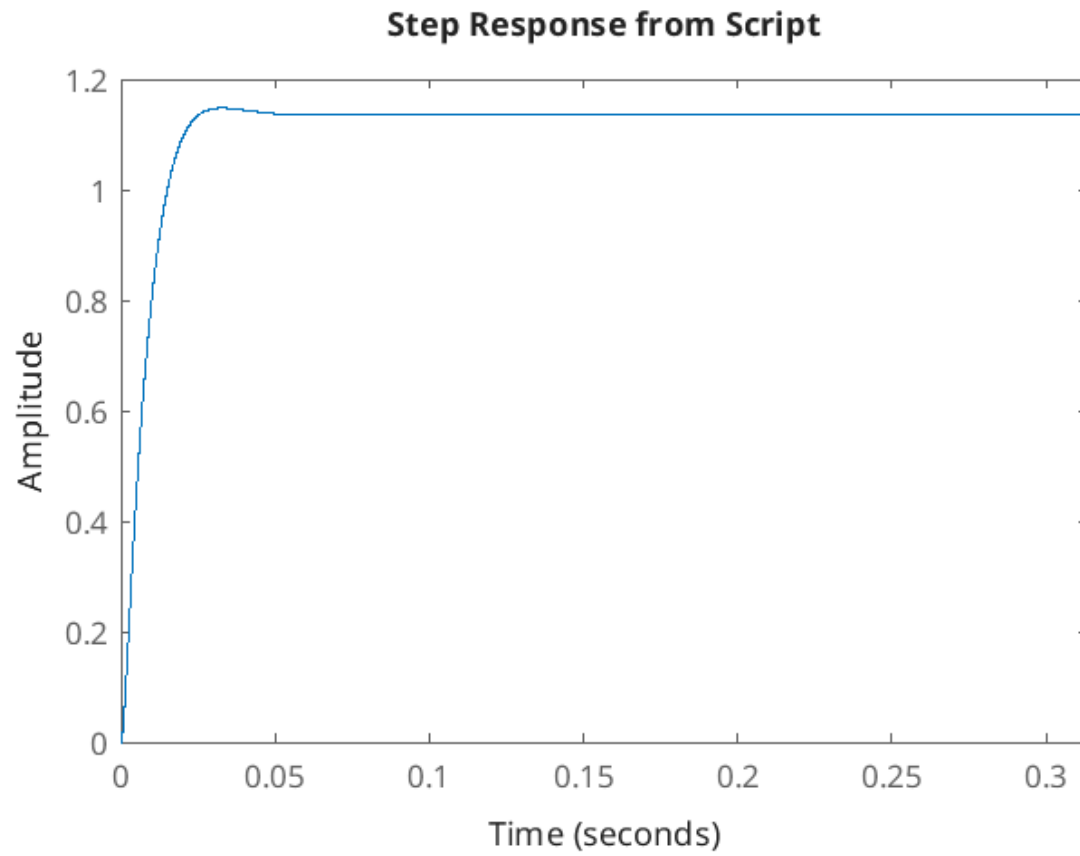
## B. Stability Figures



**Fig. 7** Bode diagram of overall system







**Fig. 9** Step response of overall system

### References

- [1] Cagliari, D., *Project Instruction Sheet*, Rensselaer Polytechnic Institute, 2025. Available from course website or instructor.